Recovering Versioned Releases Across Build and Publish Boundaries

***Abstract*—An npm package can build from its dependency's local workspace output, while an isolated registry-based Docker builder needs its base image published first. After partial failure, a consumer can also lose its pending release when its provider's success removes the original propagation reason. We present a protocol combining stage-aware scheduling with consumer-based admission of release intent. Git history determines the plan; tags record completed obligations. The protocol covers single-package repositories, monorepos, and control-repository polyrepos through a common integration contract. We formalize deterministic planning and eight conditional recovery guarantees, with counterexamples to simpler rules. All planned outcomes hold in a prospective 140-episode Docker matrix; a source-window counterfactual loses 111 obligations in 84/120 fault episodes. A 772-state finite projection finds no model violations and 260 losing states. Separate 1.10.0 evidence adds 300 reference-model agreements, 20 recovery episodes, 60 mixed-chain executions, and three CI repetitions; saved 1.8.2 observations add broader diagnostics, comparisons, and scale. The guarantees concern the model; implementation refinement and external exactly-once effects remain outside the claims.**

***Index Terms*—software release engineering, continuous integration and delivery, dependency scheduling, recovery, idempotence**

I. Introduction

Consider two npm workspace packages: a shared asset library and a UI package that imports it, followed by two Docker images. The dependent UI package consumes its provider's local build output, and the first image packages the built UI. The second image uses the first as its base; when built by an isolated registry-based builder, it must pull the new base version from a registry. Building all packages before publishing any of them is not a valid schedule for this release. Serializing every package through both stages is sufficient but also forces local-artifact consumers to wait for uploads they do not need.

Interruption makes the problem harder. If both npm packages and the base image have published but the final image fails, a repeated command should finish the remaining release. It should neither forget that image because its providers have already released nor assign a different version solely because the first attempt made progress. A release engine must therefore answer three questions: which versions are owed, which stages may run, and which obligations remain after a failure. These questions interact, but require different state.

Release engineering spans more than compilation [1]. A heterogeneous release writes to independently administered package registries, container registries, and deployment services. They provide no shared atomic commit. The resulting execution resembles a saga [2]: individual steps commit separately, and recovery must account for the work that already succeeded. For published artifacts, deletion may be unavailable or operationally unacceptable. Our protocol therefore uses forward recovery. It completes or reconciles outstanding work rather than assuming that every external effect has a compensating action.

Workflow engines can express a build/publish DAG. Existing release tools also compute versions and coordinate publication [3], [4], [5], [6], [7]. The question studied here is how to compose stage ordering with a release ledger so that successful providers do not erase their failed consumers' obligations. The key rule is consumer-based admission: a propagated unit of intent remains pending according to the consumer's own release window, even when the originating package has already recorded its release. A second distinction separates intent accumulated over a prerelease train from fresh work not yet delivered.

We present ReleaseWeave, an implementation of this protocol, and RIL, its release-intent language. These names are pseudonyms for anonymous review; reported version numbers identify the evaluated artifacts. The implementation uses Git history and tags as its release ledger and ordinary commands for build and publish stages. It requires no additional persistent release-state database. That claim concerns orchestration state, not the storage used by registries, build tools, or deployed applications.

A package is a versioned release unit: software, specifications, and documentation use the same intent, dependency, command, and completion interfaces. The protocol accommodates different build tools and registries; its guarantees require conforming integrations, not a particular language. The same graph also gives provisioned CI a stable entry point for affected tests and releases as packages evolve; configuration still records each new package, and new credentials or runners can require workflow changes.

The paper makes three contributions:

- An ecosystem-independent, saga-inspired release protocol combining local-artifact and published-artifact dependencies with explicit release intent and per-package completion records (Section II).
- A self-contained model with deterministic planning and eight recovery, discharge, and scope/channel guarantees under stated assumptions (Sections III–IV). Counterexamples explain why consumer windows and stable retry predicates matter.

- An evaluation of a released binary using differential planning, targeted vectors, injected failures, historical replay, and scale measurements (Section V), with the model-to-implementation gap made explicit.

The mathematical contribution is the admission and discharge discipline, not a new topological-sorting or Git-storage primitive.

II. System and Release Protocol

A. A Release Crossing Two Availability Boundaries

Our running example has four packages: `assets` is a shared npm library, `ui` is an npm package built from that library, `base` packages the UI output into a Docker image, and `app` builds from that image. Arrows in Fig. 1 run from provider to consumer. The second builder obtains its base through a registry; we do not assume that it shares the first builder's local image store.

Availability modes describe integration choices, not ecosystems. Here, `ui` and `base` use workspace output; `app` pulls its base from a registry. The provider setting `isBuildWaitingPublish: true` therefore belongs on `base`. Both npm providers keep the default local-build mode. Other integrations may require npm publication or share local Docker images. Consumer publication follows provider publication in both modes.

Suppose `assets` is at 1.4.2, `ui` at 3.2.0, `base` at 0.8.2, and `app` at 2.1.0. An intent to release an assets feature and patch all transitive consumers plans versions 1.5.0, 3.2.1, 0.8.3, and 2.1.1. After the first three packages publish and are recorded, `app` fails. A planner that only propagates from *currently unreleased providers* can now lose `app`, leaving an orphan in the release ledger. Our planner instead asks whether `app` has received that intent. Its own pending window is unchanged, so a retry still owes version 2.1.1.

B. From Repository Inputs to a Plan

The planner reads a fixed repository revision, package definitions, dependency edges, commit messages, and release tags. Configuration supplies build and publish commands and availability requirements; manifests can supply package and dependency identities. The plan specifies selected packages, next versions, and reasons before any external publication occurs. Changed inputs require a new plan; a preview is not an authorization for arbitrary later repository states.

Release intent extends Conventional Commits [8]. For example, `feat(assets)^^: add theme assets` directly requests a minor release of `assets` and propagates patch intent to transitive consumers. An unmarked unit affects its own scope only. A scope can explicitly name packages or be derived from changed paths. A commit can contain multiple independent units, each retaining its own source set and propagation depth. The formal model consumes normalized units, not a particular parser syntax. The complete language grammar is outside this paper [9]. A diagnostics command validates literal intent syntax and parser policy without committing; it does not verify scope or plan correctness.

Release-worthy descriptions marked during development can be collected into version-specific notes at release. Even in private projects without conventional changelogs, a human can adapt them as social announcement drafts or hand them to managers for reports. This unmeasured use does not automate posting. Notes are narratives; completion tags only discharge versioned obligations.

Version bumps and prerelease channels are separate axes; a channel transition need not request a bump. Cancellation, correction, holds, and exact versions govern pending intent without unpublishing artifacts. These controls motivate the model predicates; their CLI forms are outside our contribution.

C. Execution, Recording, and Recovery

For each selected package, the executor schedules version reconciliation, build, and publish work. A consumer's version/build work waits for provider builds, or for provider publications when their availability mode requires it. Build and publish tasks have separate concurrency limits. Independent ready tasks may proceed concurrently; deterministic planning does not require a fixed temporal order for independent completion events.

Successful publication creates a per-package completion obligation. The model records it immediately as a tag. The implementation also has configured recording and finalization steps, whose failures must be reported; their crash boundaries are not assumed atomic with an arbitrary publish command. The next invocation reconstructs outstanding work from the resulting records. This avoids a separate persistent recovery database, but depends on preserving and making the release tags available. A lock prevents cooperating release jobs from writing the same remote release concurrently. It does not make registry operations transactional or prevent an administrator from rewriting history outside the protocol.

Two failure cases must remain distinct. If a publish is known to have failed, its obligation stays pending. If the external operation succeeds but its response or completion record is lost, an absent tag cannot distinguish that success from failure. The integration must reconcile the artifact's identity or use a publisher that safely accepts repetition before recording completion. Forward recovery does not by itself solve this uncertainty [10]. The mathematical claims below refer to the recorded state and make their retry assumptions explicit.

D. Applicability and Practical Limits

The graph may contain one package: the same record-and-retry discipline still applies even though there is no propagation. A monorepo supplies several packages under one history. In a polyrepo, a control repository records submodule pointers and the release graph; its commits and tags provide the single history and ledger used by the model. The linked repositories keep their own histories. This is a construction over a control history, not an atomic transaction across independent Git repositories.

A Go consumer resolving a new module version needs its tagged source published; an upstream compiled output does

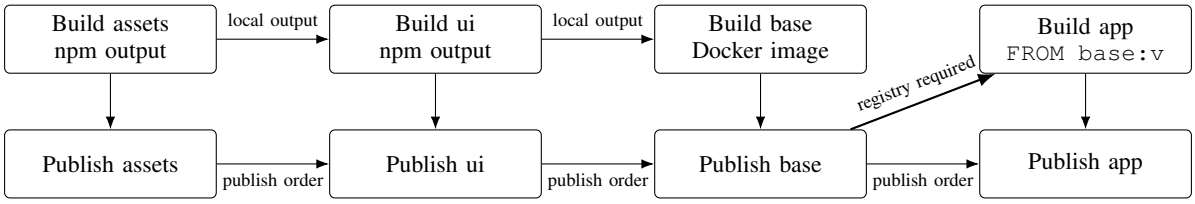


Fig. 1. Stage dependencies, not a measured timeline. Publishing `assets` may overlap building `ui`, whose publication may overlap building `base`. Building `app` must wait for `base`'s publication. Availability modes are configured requirements: an npm dependency fetched from a registry would also require publication.

not provide it. Local `go.work` or `replace` can instead bind a checkout [11]. This boundary is configuration-specific and unevaluated here.

The integration contract has four parts: stable package and version identities; declared dependencies and availability requirements; commands whose successful exit means their promised stage has completed; and durable, trustworthy completion records. Repeated publication must either be safe or reconciled when its outcome is uncertain. These requirements also admit a specification released as versioned files: validation or rendering can be its build stage, and publishing the bundle its publish stage. This is a common protocol for conforming integrations, not an assertion that every existing release workflow or version scheme fits the model. In particular, the model below assumes an acyclic graph and SemVer-compatible versions.

a) Shared release logic across operating systems: Commands permit different build ecosystems without embedding their build engines in the planner. A game developer can work on Windows while Linux CI builds a dedicated server; conversely, a Linux developer can use Windows CI for native packaging. A shared configuration holds the graph, lifecycle, and portable commands. Small Linux and Windows files reference it and select `sh` or PowerShell, with separate referenced scripts for platform-specific steps. For example, `cmake -build build` can be shared after each platform configures its own build directory. This reuses orchestration logic; it does not translate shell syntax or make build outputs portable.

Conditional and iteration commands can compose shared tasks and gates. However, in evaluated 1.10.0 the conditional wrapper itself requires a POSIX shell; a native Windows caller without it must select the platform configuration directly. Toolchains and SDKs must exist on each execution host. These are integration capabilities, not measured Windows/Linux equivalence or evidence of game-development adoption. The experiments evaluate narrower properties.

E. A Stable CI Entry Point

The same package graph can drive named tests and releases. A generic CI job invokes these operations rather than enumerating packages and dependencies. Within a provisioned environment, adding a package changes configuration and scripts but may leave the workflow unchanged. New runners, credentials, permissions, or system dependencies can still require provisioning. A configured test gate must reject missing required tests and block publication on failure; the shared graph alone is not a testing guarantee.

Affected-task execution already appears in systems such as Nx [4]. Here, one package model supports testing and recoverable release execution across artifact kinds. The two may use different windows and policies; the shared graph avoids duplicate package lists without replacing CI policy.

a) Observed self-hosting case: CI uses its checkout-built CLI for transitive test and build selection and self-release. This single-project observation shows that the protocol can drive its own repository; it is not adoption or causal productivity evidence.

III. Formal Model

This section defines the release computation and the publish protocol; Section IV proves guarantees over these definitions. The model separates stage availability from cross-run admission and discharge.

A. Stage Availability

For a selected package p, let B_p and P_p denote successful completion of its build and publish stage, respectively. A version task, when present, is a prefix of B_p. For a provider q and consumer p, the normal successful-stage dependencies include $B_q \to B_p$, $B_p \to P_p$, and $P_q \to P_p$. If q requires published-artifact availability, add $P_q \to B_p$. Here stage arrows mean prerequisite-to-successor; the package edge convention below is consumer-to-provider. The construction is over the full package graph, with stages outside the selected work treated as already satisfied only when their required artifacts are available.

Proposition 1 (stage-order safety)**.** *For a finite acyclic package graph, the constructed stage graph is acyclic. Any schedule that starts a stage only after all its prerequisites succeed respects both local-build and published-artifact availability.*

Proof. Take a provider-first topological rank of packages. Every cross-package stage edge increases that rank. Within a package, the only direction is build to publish. A directed cycle would therefore require a rank decrease or a publish-to-build edge within one package, neither of which exists. Each availability requirement is an explicit prerequisite, so a legal schedule cannot cross it early. The claim concerns ordering, not whether a command actually produces the promised artifact; that is an integration contract. □

A global build phase followed by a global publish phase violates the $P_{\text{base}} \to B_{\text{app}}$ edge in Fig. 1. Conversely, no edge requires P_{ui} to precede B_{base}, so those stages may overlap. This elementary construction uses standard topological scheduling [12], [13]; the recovery results below address the separate problem of retaining the correct selected work.

B. Workspace, State, Windows

Definition 1 (history)**.** The history is a finite DAG of commits with a distinguished $\mathsf{HEAD} = h$; $reach(c)$ is the set of commits reachable from c by parent edges, including c; write $c' \preceq c$ for $c' \in reach(c)$. h is fixed throughout. Normalization supplies a deterministic total order of the commits consistent with ancestry; "newest" refers to this order, with unit position breaking ties within a commit. Ancestry alone is only a partial order, so this ordering policy is a fixed planning input. This reading of a repository is the standard one: the commit DAG with reachability is the working model of git history in repository mining [14], and the same structure underlies formal derivations of git's three-way merge from least common ancestors [15]. Unlike the patch-theoretic accounts of version control [16], [17], [18], this model tracks no file contents, only history, tags and normalized intent. Manifests and path-derived scopes are supplied through the normalized instance rather than modeled as files.

Definition 2 (workspace)**.** $\mathcal{P}$ is the finite set of packages at h; $E \subseteq \mathcal{P} \times \mathcal{P}$ contains (a, b) when a declares a dependency on b of a configured kind; $dep(p) = \{d : (d, p) \in E\}$. The graph is acyclic; cyclic workspaces are outside this protocol and are rejected before scheduling. For $S \subseteq \mathcal{P}$, $\delta(S, d)$ is the least number of edges on a chain from a member of S to d against the arrows, ∞ if none.

Definition 3 (bumps, versions, channels)**.** $\mathbb{B} = \{\mathsf{none} < \mathsf{patch} < \mathsf{minor} < \mathsf{major}\}$ is a chain, hence a join-semilattice under $\sqcup = \max$. Versions are SemVer 2.0.0 [19] with its precedence; $ch(v)$ is stable for a non-prerelease v and otherwise v's first prerelease identifier; $core(v)$ and $cnt(v)$ are the numeric triple and the prerelease counter; $apply(v, b)$ applies a bump.

Definition 4 (state)**.** A state σ is a finite set of release tags (p, v, c) with $c \preceq h$; states are ordered by inclusion. $base_\sigma(p)$ is the highest-precedence version among p's tags, $sbase_\sigma(p)$ the highest-precedence *non-prerelease* one, $basec_\sigma(p)$ and $scom_\sigma(p)$ their commits; $ch(base_\sigma(p)) = \mathsf{stable}$ by convention when p has no tag. Equal-precedence tags with conflicting release identities are rejected. When no stable tag exists, $sbase$ is the virtual 0.0.0 and $reach(scom) = \emptyset$. An untagged package additionally has $base = 0.0.0$, $reach(basec) = \emptyset$, and a configured initial version; applying its first nonzero bump returns that initial version. Within a valid run sequence, tags are never moved or deleted, so states only grow. Rewriting a completion record is outside the guarantees. Tag write access is therefore release authority; a manifest's version field is not the release ledger. Manifests, configuration, and derived scopes remain trusted planning inputs.

Definition 5 (train and fresh-work windows)**.**

$$W_\sigma(P) \;=\; reach(h) \setminus reach(scom_\sigma(P)), \tag{1}$$

and

$$W^\circ_\sigma(P) = W_\sigma(P) \setminus reach(basec_\sigma(P)), \tag{2}$$

with an empty subtracted reach when P has no tag. W_σ is the *train window*; W°_σ is the *fresh-work window*. When P has no stable baseline, $W_\sigma(P) = reach(h)$: an unreleased package's first release accounts for its whole history. An existing project must therefore configure its initial baseline and scope mapping deliberately. The window is measured from the last *stable* tag so that a prerelease train can recompute its target over the whole train (Eq. 8). Direct admission, propagation admission and ledger discharge use W° instead. Thus a prerelease tag discharges delivered work without erasing the earlier records needed to recompute the train target.

C. Units, Suppression, Admission

Definition 6 (units)**.** $\mathcal{U}$ is the set of units carried by commits in $\bigcup_P W_\sigma(P)$, parsed and with scopes resolved and corrections applied. For $u \in \mathcal{U}$: c_u is its commit, $scope(u) \subseteq \mathcal{P}$ its resolved scope, $bump(u) \in \mathbb{B}$ the bump its type and `!` map to; its bump axis is $pb(u) \in \mathbb{B}$ (the stated value, or $bump(u)$ under `inherit`; the default once propagation is asked for is `patch` because a dependency update need not change the consumer's own API), $pd(u) \in \mathbb{N} \cup \{\infty\}$ (default 0), $ps(u)$ a scope filter; its channel axis is a channel value $pc(u)$, a depth $pcd(u)$ (default 0) and a filter $pcs(u)$.

Definition 7 (suppression and sources)**.** $canc(c, p)$ holds when c is an ancestor-or-self of a `cancel` commit whose scope contains p; $held(p)$ when p's effective `Release-As` is `none`. Both are functions of h and the configuration alone. With $disch_\sigma(p, c) \equiv c \notin W^\circ_\sigma(p)$,

$$src_\sigma(u) = \{p \in scope(u) : disch_\sigma(p, c_u) \vee \neg(canc(c_u, p) \vee held(p))\}. \tag{3}$$

Suppression applies only to *undischarged* work: once p has published the version carrying u, the artefact its consumers are owed is public, and neither a later cancel nor a later hold may retroactively strand them.

Definition 8 (reach)**.** $R(S, \ell) = \{d \in \mathcal{P} \setminus S : \delta(S, d) \leq \ell\}$. Excluding all of S means a unit never propagates to a package it bumps directly; depth is shortest-path and always measured from the originating source set, and a propagated effect never cascades its own propagation, so the depth written in the message is the whole blast radius in every run.

Definition 9 (channels)**.** $chan_\sigma(P)$, resolved for *every* package: the value proposed by the newest eligible direct `Channel` directive naming P with $c_u \in W^\circ_\sigma(P)$ and $\neg canc(c_u, P)$; else the value proposed for P by the newest eligible channel-axis unit u with $pcd(u) > 0$, $P \in R(src_\sigma(u), pcd(u)) \cap pcs(u)$, $c_u \in W^\circ_\sigma(P)$ and $\neg canc(c_u, P)$; else $ch(base_\sigma(P))$. A proposal is computed against $cur =$

$ch(base_\sigma(P))$ (the tag, never a value computed this run) and is empty when it proposes cur itself; a transition proposes only where cur matches its `<from>`; a *propagated* non-transition `stable` never graduates a prerelease package (graduation must be deliberate); `inherit` resolves to the unit's own `Channel` value if any, else to $ch(base_\sigma(q))$ for its sources q, bytewise least name deciding a disagreement. An empty proposal is ineligible; resolution chooses the newest remaining proposal, with direct proposals taking precedence over propagated ones.

Definition 10 (admission and effective bump)**.** Let $live(u)$ mean $bump(u) \neq \mathsf{none}$, $pb(u) \neq \mathsf{none}$, and $pd(u) > 0$. With $srcCh_\sigma(u)$ equal to $\{chan_\sigma(p) : p \in src_\sigma(u)\}$ and

$$rsv_\sigma(u, D) \equiv \mathsf{stable} \in srcCh_\sigma(u) \vee chan_\sigma(D) \in srcCh_\sigma(u) \tag{4}$$

(the modeled policy admits a prerelease source only on a matching consumer channel; this is an explicit release policy, not a universal claim about package installers),

$$\begin{aligned} adm_\sigma(u, D) \;\equiv\; & \\ & D \in R(src_\sigma(u), pd(u)) && \text{(reached)} \\ \wedge\; & D \in ps(u) && \text{(in scope)} \\ \wedge\; & c_u \in W^\circ_\sigma(D) && \text{(fresh window)} \\ \wedge\; & \neg\, canc(c_u, D) && \text{(not discarded)} \\ \wedge\; & live(u) && \text{(bump-live)} \\ \wedge\; & rsv_\sigma(u, D) && \text{(resolvable),} \end{aligned} \tag{5}$$

$$eff_\sigma(P) = dir_\sigma(P) \sqcup \bigsqcup \{\, pb(u) : adm_\sigma(u, P) \,\}, \tag{6}$$

where $dir_\sigma(P) = \bigsqcup\{bump(u) : P \in src_\sigma(u) \cap scope(u),\ c_u \in W^\circ_\sigma(P)\}$. Because $\sqcup$ is a join, a package with its own `feat` pending is unaffected by an incoming propagated `patch`, joining an admitted bump cannot lower the effective bump, and gathering order cannot matter.

The consumer-side staleness audit is the upward dual of Eq. 5, not merely reverse reachability. For candidate D it scans units in $W^\circ_\sigma(D)$, excludes D when it is itself a source of that unit, and applies the same shortest depth, scope, cancellation, $live$, and resolved-channel predicate rsv. These clauses make the upward answer exactly equal to the downward planner's obligations, including prerelease workspaces.

D. Plan, Versions, Runs

Definition 11 (plan and versions)**.** Write $pin_\sigma(P) = v$ when the effective surviving exact `Release-As: v` unit for P has $c_u \in W^\circ_\sigma(P)$; otherwise $pin_\sigma(P)$ is undefined. Thus a successful tag discharges a pin just like other fresh work, and the directive is fixed during a retry.

$$\begin{aligned} \Pi(\sigma) = \{\, P : \neg held(P) \wedge (eff_\sigma(P) \neq \mathsf{none} \\ \vee\; chan_\sigma(P) \neq ch(base_\sigma(P)) \\ \vee\; pin_\sigma(P)\downarrow) \,\}, \end{aligned} \tag{7}$$

each member carrying $\nu_\sigma(P)$. Let $S = sbase_\sigma(P)$, $B = base_\sigma(P)$. Define the train aggregate $train_\sigma(P)$ by Eq. 6, replacing its direct and propagated fresh-window tests $c_u \in W^\circ_\sigma(P)$ with $c_u \in W_\sigma(P)$. All other source, scope, suppression, depth, and resolved-channel predicates stay the same. It computes the target bump over the train, while eff determines fresh admission. Write $e = train_\sigma(P)$. If $pin_\sigma(P)\downarrow$, it supplies that exact version. A stable package that remains stable uses $apply(S, eff_\sigma(P))$; a prerelease package graduating to stable uses $apply(S, e)$ as required by RIL 2.0 §11.5. Otherwise, with ch the target channel,

$$\begin{aligned} & t = apply(S, e), \\ & \nu_\sigma(P) = \begin{cases} t\text{-}ch.(cnt(B)+1) & \text{if } core(B) = t \text{ and } ch(B) = ch, \\ t\text{-}ch.0 & \text{otherwise,} \end{cases} \end{aligned} \tag{8}$$

For a channel-only release whose candidate does not exceed B, recompute once with $e = \mathsf{patch}$ (the channel-entry patch); an exact pin is never adjusted this way. This permits entry from a clean stable baseline, for example 1.2.0 to 1.2.1-beta.0, without treating an actual bump regression as a channel switch. Every computed or pinned version must exceed $base_\sigma(P)$ in precedence, or the run refuses. Recomputing t from the *stable* baseline every run is what moves a whole train to `2.0.0-beta.0` when a breaking change lands mid-train (Table I).

TABLE I
EQ. 8 TRACING ONE TRAIN.

Baseline	Pending bump, channel	Next
`1.2.3`	minor, `beta`	`1.3.0-beta.0`
`1.3.0-beta.0`	no fresh commit	empty plan
`1.3.0-beta.0`	a fresh minor commit	`1.3.0-beta.1`
`1.3.0-beta.1`	minor + a new fix	`1.3.0-beta.2`
`1.3.0-beta.2`	a breaking change lands	`2.0.0-beta.0`
`1.3.0-beta.2`	none, switch to `rc`	`1.3.0-rc.0`
`1.3.0-rc.1`	none, `stable`	`1.3.0`

Definition 12 (run)**.** A run from σ is parameterised by an adversarially chosen set $F \subseteq \Pi(\sigma)$ of packages failing before recorded completion, which induces the blocked set $B = \{p \in \Pi(\sigma) \setminus F : p$ transitively depends on some $q \in F\}$, computed over the *full* workspace graph (a package with no bump this run is still a path from a dependent to a failed dependency). Publication proceeds in a topological order of the full graph, ties broken bytewise by name, filtered to the plan *last*: inducing the graph on the plan first deletes ordering paths that run through unplanned packages. In the abstract transition, each package is tagged after its own publish succeeds, never before (a tag for an unpublished version makes it unrecoverable, since Eq. 1 treats it as released), and *whatever its publish target*, including targets that upload nothing (Proposition 2). The run yields $run(\sigma, F) = \sigma \cup \{(p, \nu_\sigma(p), h) : p \in \Pi(\sigma) \setminus (F \cup B)\}$. The transition abstracts the publish-to-record interval; it does not prove that interval atomic in the implementation. A *run sequence* is $\sigma_1 \leq \sigma_2 \leq \cdots$ with $\sigma_{k+1} = run(\sigma_k, F_k)$.

The propagation bound excludes parsing, history traversal, scope resolution, and output. Those costs depend additionally on reachable commits, parent edges, and retained provenance. Cached traversal may share reachability, but must preserve each unit's source set, depth, filters, channel predicates, and identity. Merging units by directive spelling alone can change admission.

Order matters for a reason tags cannot express: a dependent can be published while its promised provider version is unavailable. If both completion tags later point at the same commit, ancestry alone does not reveal that temporal inversion. The protocol therefore prevents it through stage prerequisites. When a publish is rejected because the version already exists (a run died between publish and tag), RIL 2.0 requires content-identity verification before adoption and otherwise aborts: tagging a foreign artefact would enrol it into the release lineage. This is a protocol obligation, not a claim that ReleaseWeave can inspect arbitrary user-defined publish scripts; such integrations must supply an idempotency or identity check.

Theorem 1 (determinism)**.** $\Pi(\sigma)$*,* ν_σ*, the canonical publish order, and the version-guard refusal outcome are functions of the normalized instance* I *defined below, including its fixed history-ordering policy.*

Proof. Every definition above is a closed expression in those inputs; the three choice points (competing directives, competing propagated channels, publish-order ties) are each resolved by a stated total order: newest commit then last unit within it, and bytewise least name. □

E. Instantiation: What the Theorems Say About a Repository

A normalized instance is

$$I = (\text{history}, h, \mathcal{P}, E, \mathcal{U}, \sigma, \mathit{config}),$$

where configuration includes the fixed history-ordering policy. Repository files and messages supply these objects through normalization. The model abstracts artifact contents, arbitrary hook effects, and other configuration features. Its guarantees quantify over this fragment under acyclicity, version validation, and the run-sequence retry invariant. The implementation is not assumed to check every component of that invariant. Proving that a binary refines the model remains separate work; Section V provides tests of selected behavior, not a refinement proof.

IV. Guarantees and Proof Arguments

Cross-run claims use the *retry invariant*: the corrected tuple stream, source sets, graph, scope/depth predicates, target baselines, and resolved source/target channel admission of every outstanding unit remain unchanged, except for baseline tags written for successful packages. Two recurring components deserve names. **(H1)** says every baseline channel read by `Propagate-Channel: inherit` remains unchanged; **(H2)** says no contribution initially suppressed by cancellation, hold, or correction is later released at the same h. H1 and H2 are necessary in their respective cases but are not, by themselves, the full retry invariant. Neither is needed for one invocation's determinism.

Lemma 1 (window antitonicity)**.** *For* $\sigma' = run(\sigma, F)$*,* $W^\circ_{\sigma'}(p) = \emptyset$ *if* p *published and* $W^\circ_{\sigma'}(p) = W^\circ_\sigma(p)$ *otherwise. The train window* W *also empties after a stable release, but may remain unchanged after a prerelease release.*

Proof. The run adds only tags at h. If p gained none, both baseline commits are unchanged. If it published, $basec_{\sigma'}(p) = h$, so Eq. 2 subtracts $reach(h)$ and is empty. Only a stable release also makes $scom_{\sigma'}(p) = h$. □

Lemma 2 (source stability)**.** *Under the retry invariant,* $src_{\sigma_j}(u)$ *is constant for every outstanding unit throughout a valid run sequence.*

Proof. The corrected tuple stream and its cancellation, hold, correction, and source-set predicates are fixed components of the invariant. H2 is the component excluding later release of an initially suppressed contribution. Arbitrary tag-set inclusion outside a valid run sequence implies no such monotonicity, because a higher-version tag may point to an earlier commit. □

Lemma 3 (retry stability)**.** *Under the retry invariant, an unreleased package's resolved channel, each outstanding unit's source-channel set, and* $rsv(u, d)$ *remain constant.*

Proof. These are precisely the channel and source-set components fixed by the retry invariant. Definition 9 then selects the same direct or propagated proposal, and Eq. 4 reads the same sets. □

Remark 1. (H1) cannot be replaced by requiring stable completed releases: `inherit` reads $ch(base_\sigma(q))$, which the origin's own graduation moves: with `core` at baseline `1.5.0-beta.0` and commits `feat(core)^++1: x` and `release(core)%stable: graduate`, run 1 plans the consumer at `2.0.1-beta.0` (inherited `beta`) while `core` graduates; if the consumer fails, run 2 plans it at `2.0.1`: same package, both plans, different versions, every completed release stable. Thus unchanged inherited-channel state must be an explicit component of the retry invariant; successful stable releases alone do not imply version stability.

Theorem 2 (the eight guarantees)**.** *Write* adm_j *for* adm_{σ_j} *and* $n = |\Pi(\sigma_1)|$*. Under the retry invariant, for every unit* u*, consumer* D *and run* $j \geq 1$*:*

- G1 ***termination.*** *propagation halts after* $O(|\mathcal{U}| \cdot (|\mathcal{P}|+|E|))$ *work, whatever the declared depths.*
- G2 ***no orphans.*** $adm_1(u, D) \Rightarrow adm_j(u, D)$ *until* D *releases at a commit containing* c_u*.*
- G3 ***version stability.*** *if* $D \in \Pi(\sigma_1)$ *and* D *is not released before run* k*, then* $D \in \Pi(\sigma_k)$ *and* $\nu_k(D) = \nu_1(D)$*.*
- G4 ***no double ledger delivery.*** *once* D *releases past* c_u*,* $adm_j(u, D)$ *never holds again; with G2, ledger discharge is exactly-once for every contribution whose*

target eventually succeeds. This does not make external side effects exactly-once.

G5 ***no widening.*** $\{D : adm_j(u, D)\} \subseteq \{D : adm_1(u, D)\}$.

G6 ***convergence.*** $\Pi(\sigma_{n+1}) = \emptyset$*, provided each nonempty retry admits and publishes at least one initially publishable, non-held package.*

G7 ***channel discharge.*** *a propagated channel contribution is admitted only while its commit remains in* $W^\circ(D)$*; baseline equality and transition matching independently remove no-op proposals.*

G8 ***one-way axis independence.*** *bump-axis suppression alone does not suppress a channel change. The converse is false: resolved channels are an input to bump admission in Eq. 4.*

Proof. G1: the traversal computing R marks each package seen at most once, so it does $O(|\mathcal{P}|+|E|)$ work per unit regardless of depth or cycles; acyclicity is needed for the publish order, not here. G2: compare Eq. 5 clause by clause. Source set, depth, graph, scope, cancellation and channel admission are fixed by the retry invariant. The consumer's fresh window is unchanged until D releases (Lemma 1); that release is the only event that can make its admission clause false. G3: D gained no tag, so its baselines, windows and dir are unchanged; the propagated join is unchanged by G2 ($\geq$) and G5 ($\leq$), whose proofs below use only the lemmas; so eff is unchanged and the channel is unchanged (Lemma 3). Thus bump-only and channel-only membership remain; an exact `Release-As` remains the same because its unit is fixed by the invariant. In every case ν is the same function of unchanged inputs. G4: by Lemma 1, $W^\circ(D)$ empties after either a stable or prerelease tag, killing fresh direct and propagated admission. This also makes $pin_\sigma(D)$ undefined. This is discharge in the tag ledger; an interruption after an external publish but before its tag requires content-identity reconciliation or safe publisher repetition, not an exactly-once claim about the external operation. G5: every clause of Eq. 5 is constant except the window, which is antitone (Lemma 1); a conjunction of constant and antitone predicates is antitone. G6: $\Pi(\sigma_{j+1}) \subseteq \Pi(\sigma_j)$, since a member of the later plan was not released (else its fresh windows and pins are empty and its channel equals its new baseline) and each disjunct of Π is then unchanged or smaller; by the premise each nonempty retry removes at least one initially publishable package (G4), so $|\Pi|$ strictly decreases to 0 within n runs. The bound assumes progress on every counted nonempty run; it does not bound the number of failed attempts that make no progress. G7: the channel-axis admission test uses $c_u \in W^\circ(D)$, so Lemma 1 discharges it. While fresh, equality with the baseline and a transition's `<from>` check separately reject no-ops. G8: the channel pass runs first and reads no bump-axis output; the bump pass then reads its resolved channels through Eq. 4. □

Proposition 2 (tagging every admitted release)**.** *If a released package with no artefact to upload and a non-empty effective bump were versioned but not tagged, its window would never advance (Lemma 1), it would re-enter every plan forever, and G6 would fail. Hence Definition 12 tags whatever is released, and a fully successful model run has an empty subsequent plan. Held packages are excluded from admission. This necessity is relative to a tag-only ledger; another protocol could use other durable completion records.*

TABLE II
RUNNING EXAMPLE: PLANNED VERSIONS ASSETS 1.5.0, UI 3.2.1, BASE 0.8.3, APP 2.1.1. THIS MODEL TRACE IS NOT A TIMING MEASUREMENT. NO-PROGRESS ATTEMPTS PRESERVE SAFETY BUT DO NOT COUNT TOWARD THE CONVERGENCE BOUND.

Run	Plan	Recorded success	Failure
1	all four	assets, ui, base	app
2	app 2.1.1	app	none
3	empty	none	none

Three consequences explain the operational behavior. *Resumption*: after a partial failure, $\Pi(\sigma_2) = F \cup B$ at the original versions (G2–G5), provided the retry invariant holds. *Idempotence*: let $release(s) = run(s, \emptyset)$ denote a fully successful transformation with fixed reviewed inputs and valid records. Then

$$release(s) = release(release(s)),$$

under the retry invariant, because the first transformation discharges every admitted obligation and the second has an empty plan (G4–G6). Determinism of an unchanged preview is a separate property (Theorem 1). *Scope preservation*: G5 makes the original plan an upper bound on retry publication under the same invariant. This is not an authorization guarantee against changed commands, rewritten tags, or changed credentials.

The idempotence equation concerns recorded release state. If a publisher succeeds but its completion record is lost, its outcome must be reconciled, or the publisher must safely accept repetition. Arbitrary shell-side effects are not made exactly-once by a tag ledger.

V. EVALUATION

We ask whether consumer windows retain residual obligations where a source window loses them (RQ1); whether versions and records remain stable through failure, retry, repair, and uncertain responses (RQ2); whether execution respects artifact boundaries and CI gates (RQ3); and what planning costs appear as graph, intent, and history inputs vary (RQ4). The main evidence uses the official 1.10.0 Linux ARM64 image and implemented 2.0 release-intent semantics. Saved 1.8.2 observations remain historical.

A. Study Design and Evidence Boundaries

The prospective mechanism campaign crosses five four-package DAGs (chain, diamond, fan-out, disconnected pairs, and a chain with shortcuts), stable and beta baselines, four fault targets, and three injection boundaries: before build completion, before publication, and after an artifact is written but the test publisher returns failure. The resulting 120 fault

TABLE III
QUESTIONS, INTERVENTIONS, COUNTING UNITS, AND PRINCIPAL OUTCOMES.

RQ	Intervention	Unit	Outcome
1	finite projection; source-window counterfactual	state, transition, episode	0 projection violations; counterfactual loses 260 states and 111 obligations in 84/120 fault episodes
2	faults, repair, identity conflict	140 episodes	140 pass; 10/10 foreign artifacts refused
3	real registries; CI gates	60 executions; 3 CI repetitions	configured barriers and test gates distinguish intended pass/fail paths
4	graph/history factors	8 process cells	.017 s at 100 layered packages; 19.272 s at 10,000

episodes each repeat the fault, repair it, and run once more to test discharge; ten no-fault and ten foreign-identity controls bring the matrix to 140 episodes. The last boundary is an explicit adapter response, not a process crash. Build and publish concurrency are one; seed 20260910 fixes the order.

For RQ1, a finite model projection enumerates ordered four-node DAGs, each single origin, reachable completion sets, and failure subsets. Against the same states and observed episode completions, a computed counterfactual removes a unit when its source completes. This is a mechanism ablation, not a modified binary or competitor. For RQ2, every episode checks the residual plan against the original obligations minus completion tags, preserves target versions, and inspects immutable version/content identity. The local publisher may accept identical content after an uncertain response and must reject foreign content.

Complementary evidence uses a restricted reference model on 300 real Git histories with 5–20 packages, layered DAGs, 4–12 intent units, and mixed tag states; recovery graphs contain 20 or 100 packages. Verdaccio and Docker registries exercise the mixed chains; CI injects failing and missing tests. Historical 1.8.2 data cover a broader but imperfect oracle profile, reconstructed policy replay, pinned tool configurations, and eight scale cells. Every campaign retains its own binary, environment, and unit: model states, episodes, histories, fixture executions, and verifier predicates are reported separately. The deterministic matrices do not warrant p-values or a pooled quality score.

B. RQ1: Consumer-Window Mechanism

The finite projection covers 64 ordered four-node DAGs and four single origins: 256 source instances yield 772 reachable completion states, 2,270 failure-subset transitions, and 1,054 progress transitions. It finds no violation of exact residual obligations, no widening, no double ledger delivery, or strict decrease when progress occurs. The source-window mutant loses obligations in 260 of those states. This is an executable stable-channel, one-intent projection, not a proof of the full model.

On binary-derived completion states, the same counterfactual loses 111 obligations in 84 of the 120 fault episodes: 28/40 episodes at each of the build, prepublication, and post-write boundaries, balanced equally between stable and beta. The ten no-fault controls lose none. All ten foreign-identity controls also expose one loss each, reported separately because they test a different intervention. *Answer to RQ1.* Consumer-relative windows retain the expected residual set in the explored states; source completion is insufficient evidence that every consumer has discharged the intent.

C. RQ2: Retry and Identity Stability

All 140 mechanism episodes passed their predeclared outcomes. In the 130 fault or foreign-identity episodes, repeating the adverse condition preserved the plan, target versions, and completed tags; repair produced an empty plan. All 140 final invocations were no-ops with stable tags, and the adapter recorded one artifact write per obligation. Each of the 40 cases at each fault boundary passed, as did all 70 stable and 70 beta cases. The ten foreign artifacts were refused without a new completion tag, after which restoring the intended content allowed recovery. This supports the supplied immutable local adapter, not arbitrary registry publishers or exactly-once shell effects.

Answer to RQ2. Across the designed graph, channel, target, and fault factors, recorded progress survived repeated failure without version drift; identity reconciliation distinguished prior intended writes from conflicting content.

a) Reference agreement and specification regressions.: All 300 fresh core instances produced accepted matching plans, with no differences, refusals, or invalid execution. Of 1,500 saved full-profile histories from 1.8.2, 997 produced accepted plans and 503 produced complete E195 refusals for proposed regressive versions. Among accepted plans, 546 comparisons were clean; four refused dossiers also had no row difference, giving 550 matching diagnostic comparisons. Across the profile, 1,914 package rows differed. Rejected diagnostic output is not an executable release plan.

The 1,914 differing rows are retained for diagnosis. They show that the restricted oracle cannot support a full refinement claim, especially for prerelease and channel behavior. Six saved 1.8.2 specification vectors add selected discharge, suppression, channel, and source-exclusion cases without covering the complete language.

b) Earlier recovery and historical replay.: All 20 fresh 1.10.0 episodes made progress on every run and converged within four runs; the verifier inspected 251 command records. Saved 1.8.2 evidence contains two isolated orphan repetitions and 24 broader episodes, which also converged within four runs with zero structural G3/G5 check failures. These checks concern tested schedules and predicates. They do not establish the liveness premise under permanent registry failure, nor do they make an unrecorded external write unambiguous.

Saved 1.8.2 fault cells retain orphan and competing-push cases. The latter distinguish completed release tags from a concurrent source change; they test selected interleavings rather than a systematic concurrency sweep.

Five-release historical replay matched 36 of 42 versions in `google-cloud-node` and 7 of 64 in `conventional-changelog`. Those projects did not

use the modeled intent language. Their layouts and release policies were reconstructed for the replay, and mismatches may reflect that reconstruction as well as different release conventions. Consequently the experiment demonstrates limits of policy transfer; it is not evidence that our protocol reproduces arbitrary tools' historical version decisions.

D. Historical Comparisons and Negative Evidence

The historical refusal and competing-push fixtures pin Lerna 10.0.1, Nx 23.1.2, and Changesets 3.0.1. In those configurations, Lerna and Nx tagged the denied version before its failed upload, while Changesets left it untagged and recovered by querying npm. The observed propagation policies were unconditional for Lerna/Nx, range-gated for Changesets, and per-commit for our protocol. These are observations of the pinned configurations, not universal properties of every mode those tools expose. In competing-push fixtures, compared runs left tags outside branch ancestry; our historical run merged before its final push and planned the new commit. Release-please's tag timing comes from documentation, not an executed comparison [6]. No cross-tool speed or aggregate superiority claim follows.

E. RQ3: Availability Boundaries and CI

The fresh 1.10.0 Linux ARM64 campaign executes both the three-package npm $\rightarrow$ Docker $\rightarrow$ Docker chain and the four-package running example. Build outputs carry fixture content through OCI archives; private Verdaccio and Docker registries distinguish local output from publication [20], [21]. Each topology repeats five configurations six times in seeded order: mixed and serialized availability, a missing barrier, refused publication, and failed building.

Each topology has 30 initial executions: 12 intended successes and 18 designed failures. Per topology, 12 repairable failures completed with unchanged versions; 24 completed cases then reran without build or publication. Six missing-barrier controls were not repaired. Enabled barriers placed app building after base publication; without one, the app failed against the unavailable image. In all six mixed three-package runs, base building overlapped npm publication.

In the four-package mixed runs, UI building overlapped delayed assets publication. All six base builds finished before UI publication started. This is a valid local-output schedule: base building need not await UI publication, but overlap is not required. Intervals use a shared monotonic clock [22]. An earlier 1.9.0 attempt incorrectly required strict overlap and failed its oracle. We corrected the predicate before a full 1.9.0 rerun; 1.10.0 uses that frozen protocol. A focused oracle test accepts completion before publication starts and rejects starting after publication ends. The original failure is retained separately and is not counted as passing implementation evidence.

The three CI repetitions kept the workflow entry point unchanged while adding packages to shared configuration. They selected required tests through the graph, blocked release stages for an injected or missing test, propagated a specification change to its consumer, and observed a final no-op release rerun. These fixtures support selected ordering, recovery, and CI paths on one Linux ARM64 environment. *Answer to RQ3.* In these fixtures, configured barriers distinguish local build availability from required publication, and both failed and absent required tests prevent release; concurrency and onboarding effort are not measured outcomes.

F. RQ4: Planning Cost

Eight saved 1.8.2 cells report seconds and peak MiB. For layered graphs with 200 units, medians rise from .017/15 at 100 packages (185 edges), through .250/21 at 1,000 (2,002 edges), to 19.272/92 at 10,000 (19,919 edges). A 10,000-package chain records 19.354/118; 1,000-package chain and star cells record .254/26 and .244/19. At 1,000 layered packages, 1,000 units give .290/26 and 5,000 padding commits give .664/30. These are contended-host observations, not confidence intervals or a cross-tool benchmark. *Answer to RQ4.* Observed costs increase sharply between the 1,000- and 10,000-package cells; the design does not isolate a general asymptotic or end-to-end bound from history and output costs.

VI. Discussion and Threats to Validity

a) Meaning of the guarantees.: Safety and eventual progress are different obligations [23]. Our retry theorem fixes corrected intent, source membership, admission predicates, and relevant channel inputs. Some stability lemmas are direct consequences of that invariant. Their operational value is to expose the conditions that a retry must preserve, not to claim that any repository mutation is harmless. New commits, edits to release policy, or tag rewrites can require a new plan. A process that fails forever need not converge.

The model abstracts external artifact contents and treats successful publication followed by a completion record as one transition. Lost responses and failed recording remain integration obligations. A Git lock coordinates cooperating jobs but is not a transaction manager for arbitrary registries. The implementation's own-reason failure exceptions, recording stages, and finalization behavior require more detailed refinement than the abstract blocked-set rule. We do not claim that testing selected traces closes that gap.

b) Internal and construct validity.: The author developed the system, model, oracle, and tests, so common mistakes can survive their agreement. The restricted oracle does not cover all current semantics; full-profile categories are heuristic. The prospective mechanism matrix was frozen before its full run but designed after the earlier campaign; it is not independent replication. The finite projection covers stable-channel, one-intent states, and the post-write fault is a publisher's nonzero response, not a process crash. A passing integrity check shows consistency of supplied records. The concurrent-push cases cover selected interleavings. The measured units include histories, package rows, recovery episodes, fixture and CI repetitions, commands, and assertions; they are not pooled.

c) External validity.: Synthetic repositories and local fault fixtures establish behavior under controlled conditions. The historical subjects require reconstructed policies, and the

comparison uses pinned tool configurations. No user study establishes that authors write correct intent or that teams find it easier to review. Polyrepo applicability follows from the control-repository construction; its operational scale is not measured. Cross-platform scripts, agent gates, and multiple ecosystems demonstrate integration scope, not general effectiveness. Performance observations from one contended host are not service-level promises.

VII. Related Work

a) Release and build systems.: Adams and McIntosh identify release engineering as a distinct research problem [1]. Large monorepo practice [24] and build-system models [13] address dependency ordering and reuse. Incremental build work also formalizes dynamic dependencies [25]; our model instead uses a fixed release graph and records versioned publication obligations. We claim neither incremental rebuilding nor a novel topological algorithm. Lerna, Nx, Changesets, release-please, and semantic-release provide different versioning and publication workflows [3], [4], [5], [6], [7]. Section V-D compares operational traces of specified configurations, not formal reductions of those complete tools. Spinnaker already supports parallel pipeline stages and execution histories [26]; expressing a DAG is not unique to this protocol. Nix identifies component instances through input-derived store paths [27], while TOSCA standardizes service templates and lifecycle interfaces [28]. OpenGitOps calls for versioned desired state and continuous reconciliation [29]. Our ledger instead tracks completed versioned releases across independent publication targets. We neither replace these systems' state models nor claim their guarantees for arbitrary scripts.

Version intent is not a proof of API compatibility. Studies of versioning and ecosystem change [30], [31], [32] motivate explicit policy but do not show that a declared bump is correct. Package-installability work such as EDOS [33] models a different consistency question: a set of installable packages rather than the remaining obligations of an interrupted publication. Huang et al. show that measured CI effects vary with project composition [34]; this constrains generalization from our controlled fixtures rather than validating a target population. Cardoen et al. document workflow duplication despite GitHub Actions reuse mechanisms [35]; that evidence motivates shared configuration, but we do not measure a maintenance improvement. Formal VCS work studies contents, patches, and merges [16], [17], [18], [15]; Git-mining work explains why history and reference interpretation need care [14]. Our ledger uses those repository objects while leaving content and merge correctness outside its proofs.

b) Recovery and formal assurance.: Platform-independent deployment models already formalize lifecycle and version compatibility: Liu and Smith's buildbox includes build, install, ship, and update transitions [36]. Aeolus studies component states, dependencies, and configuration reachability [37]. These are direct antecedents to a common lifecycle abstraction. Our question is narrower: preserving versioned publication obligations across partial executions. The NEPHELE orchestration report also explicitly uses saga patterns for distributed lifecycle operations [38]; applying sagas to operations is therefore not itself a novelty claim. Bui et al. evaluate LLM-assisted failure diagnosis and repair in one industrial CI/CD pipeline using a knowledge ablation [39]. Our mechanism instead retains remaining versioned obligations; the tasks and outcome measures are not comparable. Sagas [2] and the limits of transaction abstractions [40] provide the recovery setting. Independently committed and idempotently processable operations [41], [10] explain why uncertain outcomes need explicit treatment. Our contribution specializes admission and discharge to versioned consumers; it is not the invention of forward recovery. Anvil [42] verifies reconciliation properties of Kubernetes controller implementations, whereas our handwritten proofs establish a release model and our tests sample implementation behavior. This is a significant difference in assurance. Methodological cautions about fuzz evaluation [43] likewise motivate separating core agreement from unsupported generalization.

VIII. Conclusion

Local-artifact consumers may proceed after a provider builds; registry-artifact consumers must wait for publication. Recovery additionally requires a failed consumer to retain its release obligation after successful providers leave the plan. Under the retry invariant, consumer-based admission with separate train and fresh-work windows preserves versions and permits convergence as recorded work progresses. The mechanism matrix retained residual plans and versions in all 140 designed episodes, while the source-window counterfactual lost work in 84 of 120 fault episodes. Real-registry fixtures support the availability distinction; restricted-oracle and historical-policy mismatches bound broader claims. Mechanized proofs and implementation refinement, including recording failures and randomized interleavings, remain open [44].

Data Availability

Campaign records are retained locally with pinned runtime identities, harness code, raw observations, hash-verified archives, and offline verifiers, including fixture Git metadata. No public anonymous replication package, external upload, or public artifact availability is claimed.

AI Assistance Disclosure

Most AI assistance used Claude Fable 5; Claude Opus 5, Claude Fable 5.1, Gemini, and OpenAI Codex also aided implementation, writing, and experiments beyond editing. Executions produced the measurements; the human author remains responsible.

References

[1] B. Adams and S. McIntosh. Modern release engineering in a nutshell: Why researchers should care. In *Proceedings of the 23rd IEEE International Conference on Software Analysis, Evolution, and Reengineering (SANER)*, vol. 5, pages 78–90, 2016. https://doi.org/10.1109/saner.2016.108.

[2] H. Garcia-Molina and K. Salem. Sagas. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*, pages 249–259, 1987. https://doi.org/10.1145/38713.38742.

[3] Lerna: a fast, modern build system for managing and publishing multiple JavaScript packages from the same repository. https://lerna.js.org/, accessed September 9, 2026.

[4] Nx: smart monorepos, fast CI. https://nx.dev/, accessed September 9, 2026.

[5] Changesets: a way to manage your versioning and changelogs. https://github.com/changesets/changesets, accessed September 9, 2026.

[6] release-please: automated releases based on conventional commits. https://github.com/googleapis/release-please, accessed September 9, 2026.

[7] semantic-release: fully automated version management and package publishing. https://github.com/semantic-release/semantic-release, accessed September 9, 2026.

[8] Conventional Commits 1.0.0. https://www.conventionalcommits.org/en/v1.0.0/, accessed September 9, 2026.

[9] RIL release-intent specification, version 2.0.0 (review pseudonym). Public source URL omitted for anonymous review, accessed September 9, 2026.

[10] P. Helland. Idempotence is not a medical condition. *Communications of the ACM*, 55(5):56–65, 2012. https://doi.org/10.1145/2160718.2160734.

[11] The Go Authors. Publishing a Module; Go Modules Reference. https://go.dev/doc/modules/publishing; https://go.dev/ref/mod, accessed September 10, 2026.

[12] A. B. Kahn. Topological sorting of large networks. *Communications of the ACM*, 5(11):558–562, 1962. https://doi.org/10.1145/368996.369025.

[13] A. Mokhov, N. Mitchell, and S. Peyton Jones. Build systems à la carte. *Proceedings of the ACM on Programming Languages*, 2(ICFP):79:1–79:29, 2018. https://doi.org/10.1145/3236774.

[14] C. Bird, P. C. Rigby, E. T. Barr, D. J. Hamilton, D. M. Germán, and P. Devanbu. The promises and perils of mining Git. In *Proceedings of the 6th IEEE International Working Conference on Mining Software Repositories (MSR)*, pages 1–10, 2009. https://doi.org/10.1109/msr.2009.5069475.

[15] G. Kaki, S. Priya, K. C. Sivaramakrishnan, and S. Jagannathan. Mergeable replicated data types. *Proceedings of the ACM on Programming Languages*, 3(OOPSLA), Article 154, 2019. https://doi.org/10.1145/3360580.

[16] W. Swierstra and A. Löh. The semantics of version control. In *Proceedings of the ACM International Symposium on New Ideas, New Paradigms, and Reflections on Programming & Software (Onward!)*, pages 43–54, 2014. https://doi.org/10.1145/2661136.2661137.

[17] C. Angiuli, E. Morehouse, D. R. Licata, and R. Harper. Homotopical patch theory. In *Proceedings of the 19th ACM SIGPLAN International Conference on Functional Programming (ICFP)*, pages 243–256, 2014. https://doi.org/10.1145/2628136.2628158.

[18] S. Mimram and C. Di Giusto. A categorical theory of patches. *Electronic Notes in Theoretical Computer Science*, 298:283–307, 2013. https://doi.org/10.1016/j.entcs.2013.09.018.

[19] T. Preston-Werner. Semantic Versioning 2.0.0. https://semver.org/spec/v2.0.0.html, accessed September 9, 2026.

[20] Open Container Initiative. *Open Container Initiative Image Format Specification*, version 1.1.0. Image manifest, configuration, filesystem layers, and image layout. https://github.com/opencontainers/image-spec/blob/v1.1.0/spec.md, accessed September 9, 2026.

[21] Moby Project Contributors. *BuildKit: Concurrent, Cache-Efficient Build Graph Solver*. Documentation of container image, OCI archive, and registry exporters. https://github.com/moby/buildkit, accessed September 9, 2026.

[22] Python Software Foundation. *Python 3.12 Library Reference: Time Access and Conversions*. Monotonic clocks and cross-process clock behavior on macOS. https://docs.python.org/3.12/library/time.html, accessed September 9, 2026.

[23] B. Alpern and F. B. Schneider. Defining liveness. *Information Processing Letters*, 21(4):181–185, 1985. https://doi.org/10.1016/0020-0190(85)90056-0.

[24] R. Potvin and J. Levenberg. Why Google stores billions of lines of code in a single repository. *Communications of the ACM*, 59(7):78–87, 2016. https://doi.org/10.1145/2854146.

[25] S. Erdweg, M. Lichter, and M. Weiel. A sound and optimal incremental build system with dynamic dependencies. In *Proceedings of the 2015 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, pages 89–106, 2015. https://doi.org/10.1145/2814270.2814316.

[26] Spinnaker Authors. *Pipelines: Stages, Parallel Paths, and Execution History*. Spinnaker documentation. https://spinnaker.io/docs/concepts/pipelines/, accessed September 9, 2026.

[27] E. Dolstra, M. de Jonge, and E. Visser. Nix: A safe and policy-free system for software deployment. In *Proceedings of the 18th Large Installation System Administration Conference (LISA)*, pages 79–92, USENIX Association, 2004. https://www.usenix.org/legacy/events/lisa04/tech/dolstra.html.

[28] M. Rutkowski, C. Lauwers, C. Noshpitz, and C. Curescu, editors. *TOSCA Simple Profile in YAML Version 1.3*. OASIS Standard, February 26, 2020. https://docs.oasis-open.org/tosca/TOSCA-Simple-Profile-YAML/v1.3/os/TOSCA-Simple-Profile-YAML-v1.3-os.html.

[29] OpenGitOps Working Group. *GitOps Principles*, version 1.0.0. Declarative, versioned and immutable, pulled automatically, and continuously reconciled desired state. https://opengitops.dev/, accessed September 9, 2026.

[30] S. Raemaekers, A. van Deursen, and J. Visser. Semantic versioning versus breaking changes: A study of the Maven repository. In *Proceedings of the 14th IEEE International Working Conference on Source Code Analysis and Manipulation (SCAM)*, pages 215–224, 2014. https://doi.org/10.1109/scam.2014.30.

[31] C. Bogart, C. Kästner, J. Herbsleb, and F. Thung. How to break an API: Cost negotiation and community values in three software ecosystems. In *Proceedings of the 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*, pages 109–120, 2016. https://doi.org/10.1145/2950290.2950325.

[32] A. Decan, T. Mens, and P. Grosjean. An empirical comparison of dependency network evolution in seven software packaging ecosystems. *Empirical Software Engineering*, 24(1):381–416, 2019. https://doi.org/10.1007/s10664-017-9589-y.

[33] F. Mancinelli, J. Boender, R. Di Cosmo, J. Vouillon, B. Durak, X. Leroy, and R. Treinen. Managing the complexity of large free and open source package-based software distributions. In *Proceedings of the 21st IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 199–208, 2006. https://www.dicosmo.org/Articles/2006-Edos-Ase.pdf.

[34] S. Huang, A. Zaidman, and S. Proksch. Does one CI-ze fit all? How continuous integration performs in different contexts. In *Proceedings of the 33rd IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pages 491–502, 2026. https://doi.org/10.1109/SANER67736.2026.00060.

[35] G. Cardoen, T. Mens, and A. Decan. An empirical analysis of code clones in GitHub Actions workflows. In *Proceedings of the 33rd IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pages 170–180, 2026. https://doi.org/10.1109/SANER67736.2026.00028.

[36] Y. D. Liu and S. F. Smith. A formal framework for component deployment. In *Proceedings of the 21st Annual ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA)*, pages 325–344, 2006. https://pl.cs.jhu.edu/projects/new-arch/papers/a-formal-framework-for-component-deployment.pdf. https://doi.org/10.1145/1167473.1167502.

[37] R. Di Cosmo, J. Mauro, S. Zacchiroli, and G. Zavattaro. Aeolus: A component model for the cloud. *Information and Computation*, 239:100–121, 2014. https://doi.org/10.1016/j.ic.2014.11.002.

[38] NEPHELE Consortium. D4.1: Initial release of hyper-distributed applications synergetic meta-orchestration framework, development environment and repository. Public project deliverable, version 1.0, December 27, 2023, pages 38–41. https://nephele-project.eu/sites/nephele/files/public/content-files/deliverables/nephele_d4.1.pdf.

[39] D. Bui, S. Grintz, A. Berndt, and T. Bach. Using large language models to support automation of failure management in CI/CD pipelines: A case study in SAP HANA. In *Proceedings of the 33rd IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pages 524–535, 2026. https://doi.org/10.1109/SANER67736.2026.00063.

[40] J. Gray. The transaction concept: Virtues and limitations. In *Proceedings of the 7th International Conference on Very Large Data Bases (VLDB)*, pages 144–154, 1981. https://www.vldb.org/conf/1981/P144.PDF.

[41] P. Helland. Life beyond distributed transactions: an apostate's opinion. In *Proceedings of the 3rd Biennial Conference on Innovative Data Systems Research (CIDR)*, pages 132–141, 2007. https://www.cidrdb.org/cidr2007/papers/cidr07p15.pdf.

[42] X. Sun, W. Ma, J. T. Gu, Z. Ma, T. Chajed, J. Howell, A. Lattuada, O. Padon, L. Suresh, A. Szekeres, and T. Xu. Anvil: Verifying liveness of cluster management controllers. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 649–666, 2024. https://www.usenix.org/conference/osdi24/presentation/sun-xudong.

[43] G. Klees, A. Ruef, B. Cooper, S. Wei, and M. Hicks. Evaluating fuzz testing. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 2123–2138, 2018. https://doi.org/10.1145/3243734.3243804.

[44] L. Lamport. *Specifying Systems: The TLA$^+$ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002. https://lamport.azurewebsites.net/tla/book.html.